

Trabalho Prático – Etapa 2 – GCC130

Analizador Sintático (*Bison*)

Caio Bueno Finocchio Martins

Lana da Silva Miranda

Nina Tobias Novikoff da Cunha Ribeiro

Universidade Federal de Lavras (UFLA)

Professor: Ricardo Terra

1 Introdução

A análise sintática constitui a segunda etapa do processo de compilação, na qual a sequência de *tokens* gerada pelo analisador léxico é verificada quanto à conformidade com a gramática da linguagem. Utiliza-se, para isso, uma gramática livre de contexto em notação BNF (*Backus-Naur Form*), que define formalmente construções válidas, incluindo expressões, declarações, comandos e blocos estruturais.

Nesta etapa do projeto, implementou-se um analisador sintático utilizando *Bison*, que organiza os elementos do código-fonte em construções válidas segundo a gramática da minilinguagem previamente definida. O analisador reconhece declarações de variáveis e funções, comandos de entrada e saída, estruturas condicionais e de repetição, além de expressões aritméticas, lógicas e relacionais. A implementação inclui a separação entre comandos abertos e fechados, permitindo tratar ambiguidades clássicas, como o *dangling else*, e a utilização do mecanismo de *panic mode* para recuperação de erros sintáticos.

Dessa forma, o analisador sintático desenvolvido estabelece a base para a interpretação correta do programa, garantindo que o código esteja estruturalmente válido e pronto para as etapas seguintes do compilador.

2 Gramática em BNF

A seguir é apresentada a gramática completa da linguagem, especificada em notação BNF. As produções estão agrupadas tematicamente para facilitar a leitura.

Gramática em BNF

```
<S> ::= <elementos>
```

```
<elementos> ::= <elementos> <elemento>  
              | <elemento>
```

```
<elemento> ::= <declaracao_funcao>
```

```

| <comando>

<tipo_dado> ::= TD_INTEGER | TD_FLOAT | TD_BOOL

<declaracao_variavel> ::= <tipo_dado> <lista_declaracao_variavel> ';'

<lista_declaracao_variavel> ::= <lista_declaracao_variavel> ', '
<item_declaracao_variavel>
    | <item_declaracao_variavel>

<item_declaracao_variavel> ::= ID
    | ID OP_ATRIBUICAO <expressao>

<atribuicao> ::= ID OP_ATRIBUICAO <expressao>

<declaracao_funcao> ::= <tipo_dado> ID '(' <parametros> ')' <bloco>

<parametros> ::= <lista_parametros>
    | ε

<lista_parametros> ::= <lista_parametros> ', ' <parametro>
    | <parametro>

<parametro> ::= <tipo_dado> ID

<chamada_funcao> ::= ID '(' <argumentos> ')',

<argumentos> ::= <lista_argumentos>
    | ε

<lista_argumentos> ::= <lista_argumentos> ', ' <expressao>
    | <expressao>

<bloco> ::= '{' <comandos> '}',

<comandos> ::= ε
    | <comandos> <comando>

<comando> ::= <comando_fechado>
    | <comando_aberto>

<comando_fechado> ::= <atribuicao> ';'
    | <declaracao_variavel>
    | <chamada_funcao> ';'
    | <bloco>
    | <comando_saida>
    | <comando_entrada>
    | <comando_retorno>
    | <matched_if>
    | <comando_repeticao_fechado>

```

```

<matched_if> ::= PR_IF '(' <expressao> ')' <comando_fechado> PR_ELSE
<comando_fechado>

<comando_aberto> ::= <open_if>
| <comando_repeticao_aberto>

<open_if> ::= PR_IF '(' <expressao> ')' <comando>
| PR_IF '(' <expressao> ')' <comando_fechado> PR_ELSE <comando_aberto>

<comando_repeticao_fechado> ::= PR_WHILE '(' <expressao> ')'
<comando_fechado>

<comando_repeticao_aberto> ::= PR_WHILE '(' <expressao> ')' <comando_aberto>

<comando_saida> ::= PR_PRINT '(' <expressao> ')' ';'

<comando_entrada> ::= PR_READ '(' ID ')' ';'

<comando_retorno> ::= PR_RETURN <expressao> ';'
| PR_RETURN ';'

<expressao> ::= <expressao_ou>

<expressao_ou> ::= <expressao_ou> OL_OR <expressao_e>
| <expressao_e>

<expressao_e> ::= <expressao_e> OL_AND <expressao_not>
| <expressao_not>

<expressao_not> ::= OL_NOT <expressao_relacional>
| <expressao_relacional>

<expressao_relacional> ::= <expressao_aritmetica> <operador_relacional>
<expressao_aritmetica>
| <expressao_aritmetica>

<operador_relacional> ::= OR_LT | OR_GT | OR_EQ | OR_LE | OR_GE | OR_NE

<expressao_aritmetica> ::= <expressao_aritmetica> OA_PLUS <expressao_termo>
| <expressao_aritmetica> OA_MINUS <expressao_termo>
| <expressao_termo>

<expressao_termo> ::= <expressao_termo> OA_MULT <expressao_fator>
| <expressao_termo> OA_DIV <expressao_fator>
| <expressao_fator>

<expressao_fator> ::= '(' <expressao> ')'
| <chamada_funcao>
| ID | NUMERO_INT | NUMERO_FLOAT | LITERAL
| PR_TRUE | PR_FALSE

```

3 Decisões de Projeto

1. Identificação única de identificadores

Para que cada ocorrência do *token* ID fosse corretamente referenciada na tabela de símbolos, inicialmente cogitou-se utilizar o valor do *hash* como número de referência. Entretanto, como a tabela *hash* utilizada na etapa anterior possuía listas encadeadas, poderiam ocorrer colisões com múltiplos *tokens* compartilhando o mesmo *hash*. Embora uma alternativa fosse migrar para uma tabela de endereçamento aberto, optou-se por implementar um identificador sequencial único para cada *token* ID, garantindo que a referência ficasse mais legível e mantendo a integridade da tabela sem alterar a estrutura já estabelecida.

2. Tokens específicos e uso de *yylval*

Na etapa anterior, adotou-se o padrão de *tokens* genéricos para categorias, como OPERADOR_LOGICO, diferenciando os valores específicos pelo *yylval* (por exemplo, OL_NOT, OL_OR, OL_AND). No entanto, para a integração com o *Bison*, manter esse padrão revelou-se inviável, pois o *parser* reconhece de forma mais eficiente *tokens* individuais, o que reduz ambiguidades e facilita a escrita da gramática. Assim, no *Flex*, os *tokens* foram redefinidos para que apenas os *tokens* com valor semântico utilizassem *yylval*, como ID, números inteiros e ponto flutuante, e literais.

3. Delimitadores

Para os *tokens* da categoria “delimitador” (;, {, }, (,), ,), optou-se por retornar diretamente o caractere correspondente no *Flex*, em vez de criar *tokens* separados. Essa abordagem foi apresentada nas aulas, mantém a legibilidade da gramática e simplifica o reconhecimento de blocos e expressões no *Bison*.

4. Estrutura de expressões e precedência

A gramática foi organizada em níveis de expressões para refletir a precedência dos operadores. Os níveis mais internos possuem prioridade maior, sendo avaliados primeiro:

```
1 expressao_ou -> expressao_e -> expressao_not -> expressao_relacional
2             -> expressao_aritmetica -> expressao_termo -> expressao_fator
```

Listagem 1: Hierarquia de precedência das expressões

Exemplo de avaliação para a expressão `trueB || !falseA && x < 10`:

- Primeiro, a comparação `x < 10` (*expressao_relacional*);
- Em seguida, a negação `!falseA` (*expressao_not*);
- Depois, a operação `&&` (*expressao_e*);
- Finalmente, a operação `||` (*expressao_ou*).

No nível aritmético, multiplicações e divisões possuem prioridade sobre adições e subtrações, garantindo a ordem correta de avaliação. O operador unário negativo (`UMINUS`) foi tratado no nível mais interno da hierarquia de expressões, garantindo que seja avaliado antes de qualquer operador binário. Não foi necessário usar diretivas `%left` ou `%right`, pois a estrutura da gramática já impõe a precedência correta por meio da ordem dos não-terminais.

5. Tratamento do *dangling else*

Inicialmente, foi adotada a estratégia de `matched_if` e `open_if` sem obrigatoriedade de chaves, conforme discutido em aula. Entretanto, em uma gramática maior, essa abordagem gerou conflitos, pois comandos condicionais poderiam conter laços, que também poderiam conter condicionais aninhados, tornando ambígua a associação do `else`.

Uma versão da gramática que obrigava o uso de chaves para as estruturas condicionais e de repetição resolvia esses conflitos, mas posteriormente buscamos uma solução que eliminaria as ambiguidades sem a necessidade de chaves. Para tal, os comandos foram classificados em:

- **Comandos fechados:** estruturas completas, como atribuições, chamadas de função ou blocos delimitados, cujo escopo é claramente definido;
- **Comandos abertos:** `open_if` e laços que podem conter outras construções condicionais abertas.

Essa estratégia manteve a completude da gramática, incluindo todas as combinações de comandos necessárias, e eliminando os conflitos de redução/*shift*.

6. Modo pânico estruturado com `yyerror`

Decidiu-se pela implementação de regras de erro vinculadas ao delimitador ponto e vírgula (`error ' ; '`), integradas à macro `yyerror`. Essa abordagem força o *Bison* a resetar seu estado de pânico imediatamente após localizar o término da instrução malformada, permitindo que o compilador continue a leitura e liste múltiplos erros sintáticos em uma única execução.

7. Mensagens de erro e nomeação de `%tokens`

Para tornar a depuração mais eficiente, optou-se por aprimorar a função `yyerror` e definir aliases legíveis para os `%tokens` diretamente no *Bison*. Ao invés de apresentar códigos técnicos genéricos, o compilador passou a emitir mensagens mais precisas e amigáveis, explicando o contexto do erro no padrão estruturado de elementos inesperados e esperados (por exemplo: unexpected “+”, expected “”).

4 Dificuldades Encontradas

- **Integração do *Flex* com *Bison*:** Uma das dificuldades iniciais foi entender como transferir corretamente os tokens do *Flex* para o *Bison* e usar `yyval` apenas para tokens com valor semântico. Foi necessário adaptar grande parte do arquivo *Flex*. Além disso, surgiram dúvidas sobre como invocar `yyparse()` sem comprometer a impressão da tabela de símbolos.

- **Conflitos de gramática:** Durante o desenvolvimento do analisador sintático, foram identificados conflitos que exigiram atenção detalhada. Além dos conflitos provocados pelo *dangling else*, nas primeiras versões da gramática era recorrente haver conflitos em regras que permitiam tanto múltiplos comandos sequenciais quanto uma única unidade de comando. Para resolver essa ambiguidade, a gramática foi reorganizada de forma a definir claramente uma lista de comandos que poderia ser vazia ou conter múltiplos comandos, garantindo um ponto de parada para a redução. Exemplos incluem os estados `parametros` e `lista_parametros`:

```

1 parametros:
2     lista_parametros
3     | /* vazio */
4 ;
5
6 lista_parametros:
7     lista_parametros ',' parametro
8     | parametro
9 ;

```

- **Separação entre responsabilidades da etapa sintática e semântica:** Surgiu a dúvida acerca da participação da análise sintática no tratamento de tipo das variáveis na tabela de símbolos, mas essa responsabilidade foi atribuída exclusivamente à etapa semântica.
- **Recuperação de múltiplos erros sintáticos:** A principal barreira técnica consistiu em evitar que o analisador sintático abortasse a execução imediatamente após a detecção da primeira falha. Compreender o comportamento padrão do *Bison* e configurar adequadamente o "modo pânico" exigiu o mapeamento de como o *parser* descarta *tokens* e busca pontos de sincronização, impedindo que o compilador travasse ou gerasse uma avalanche de erros falsos em cascata.

5 Conjuntos FIRST e FOLLOW

Esta seção apresenta os conjuntos FIRST e FOLLOW calculados para os não-terminais relacionados às expressões da gramática. Esses conjuntos são utilizados na construção de analisadores preditivos e na verificação de conflitos em analisadores LALR(1).

Conjunto FIRST

$$FIRST(expressao) = \{ OL_NOT, (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$$

$$FIRST(expressao_ou) = \{ OL_NOT, (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$$

$$FIRST(expressao_e) = \{ OL_NOT, (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$$

$FIRST(expressao_not) = \{ OL_NOT, (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$

$FIRST(expressao_relacional) = \{ (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$

$FIRST(expressao_aritmética) = \{ (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$

$FIRST(expressao_termo) = \{ (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$

$FIRST(expressao_fator) = \{ (, ID, NUMERO_INT, NUMERO_FLOAT, LITERAL, PR_TRUE, PR_FALSE, OA_MINUS \}$

Conjunto FOLLOW

$FOLLOW(expressao) = \{), ,, ; \}$

$FOLLOW(expressao_ou) = \{ OL_OR,), ,, ; \}$

$FOLLOW(expressao_e) = \{ OL_AND, OL_OR,), ,, ; \}$

$FOLLOW(expressao_not) = \{ OL_AND, OL_OR,), ,, ; \}$

$FOLLOW(expressao_relacional) = \{ OL_AND, OL_OR,), ,, ; \}$

$FOLLOW(expressao_aritmética) = \{ OR_LT, OR_GT, OR_EQ, OR_LE, OR_GE, OR_NE, OA_PLUS, OA_MINUS, OL_AND, OL_OR,), ,, ; \}$

$FOLLOW(expressao_termo) = \{ OA_MULT, OA_DIV, OR_LT, OR_GT, OR_EQ, OR_LE, OR_GE, OR_NE, OA_PLUS, OA_MINUS, OL_AND, OL_OR,), ,, ; \}$

$FOLLOW(expressao_fator) = \{ OA_MULT, OA_DIV, OR_LT, OR_GT, OR_EQ, OR_LE, OR_GE, OR_NE, OA_PLUS, OA_MINUS, OL_AND, OL_OR,), ,, ; \}$

6 Estados Conflitantes do Autômato

Na versão final da gramática, não foram identificados conflitos do tipo *shift/reduce* ou *reduce/reduce* durante a geração do analisador sintático pelo *Bison*. Dessa forma, não há estados conflitantes ativos no autômato final que precisem ser resolvidos por regras adicionais de precedência, associatividade ou escolha manual de redução.

Apesar disso, alguns pontos da gramática apresentavam potencial para gerar conflitos durante o desenvolvimento. O principal caso foi o problema do *dangling else*, em que um comando `else` pode ser ambigualmente associado a mais de um comando `if` em estruturas condicionais aninhadas. Conforme discutido na seção de decisões de projeto, essa situação foi tratada por meio da separação entre comandos fechados e comandos abertos, representados por produções como `matched_if` e `open_if`. Essa organização permite que o `else` seja associado corretamente ao `if` mais próximo ainda não fechado, eliminando a ambiguidade estrutural.

Outro ponto que poderia causar conflitos está relacionado à precedência dos operadores aritméticos, relacionais e lógicos, especialmente no caso do operador -, que pode aparecer como operador binário de subtração ou como operador unário negativo. Para evitar ambiguidades, a gramática foi organizada em níveis de expressão, separando `expressao_ou`, `expressao_e`, `expressao_not`, `expressao_relacional`, `expressao_aritmetica`, `expressao_termo` e `expressao_fator`. Dessa forma, a precedência é definida diretamente pela estrutura das produções, dispensando declarações explícitas de associatividade.

Portanto, embora houvesse pontos críticos durante a elaboração da gramática, a versão final apresentada neste relatório não possui conflitos sintáticos reportados pelo *Bison*. As possíveis fontes de conflito foram tratadas na própria modelagem gramatical, garantindo maior clareza, previsibilidade e consistência ao analisador sintático.

7 Arquivo de Teste e Saída Esperada

7.1 Arquivo de Entrada (teste_lexer.txt)

```
1 // Teste válido para o analisador sintático
2
3 int $x = -5;
4 float $y = 3.14;
5 bool $ativo = true;
6
7 int $soma(int $a, int $b) {
8     int $resultado;
9     $resultado = $a + $b;
10    return $resultado;
11 }
12
13 bool $maiorQueZero(int $n) {
14     if ($n > 0)
15         return true;
16     else
17         return false;
18 }
19
20 $x = $soma(2, 3);
21 $y = $y + 1.5;
22
23 if ($x > 3)
24     print($x);
25 else
26     print($y);
27
28 if ($ativo && $x != 0) {
29     print("ativo");
30     read($x);
31 }
32
33 while ($x > 0)
34     $x = $x - 1;
35
36 while ($x < 10) {
```



```

37     $x = $x + 1;
38     print($x);
39 }
40
41 if ($x > 0)
42     if ($y > 1.0)
43         print($y);
44     else
45         print($x);
46
47 if ($x == 0)
48     while ($y > 0.0)
49         if ($ativo)
50             print("loop");
51         else
52             read($x);
53
54 /* Comentário de
55    múltiplas linhas válido */
56
57 // =====
58 // TESTES PARA ERROS SINTÁTICOS
59 // =====
60 // Nota: O Lexer vai ler tudo perfeitamente, mas o Bison vai acusar erros.
61
62 // 1. Operadores duplicados / Expressão malformada
63 // O parser espera um identificador ou número após o '+', mas encontra um '*'.
64 $x = $a + * $b;
65
66 // 2. Parênteses desbalanceados em expressões
67 // Faltou fechar o parêntese antes do ponto e vírgula.
68 $y = (2 + 3;
69
70 // 3. Estrutura do comando 'if' malformada (faltando parênteses na condição)
71 // Na linguagem C/Java, o parêntese em volta da expressão do if é obrigatório.
72 if $x > 0
73     print($x);
74
75 // 4. Erro em parâmetros de função (faltando vírgula)
76 // O parser espera um ',' ou ')' após o primeiro parâmetro, mas encontra 'int'.
77 int $funcaoErro(int $param1 int $param2) {
78     return 0;
79 }
80
81 // 5. Atribuição invertida (L-value inválido)
82 // A gramática exige que o lado esquerdo do '=' seja uma variável (ID), não um número.
83 5 = $x;
84
85 // 6. Faltando a expressão dentro de um comando de repetição
86 // Um laço while não pode ter a condição vazia.
87 while () {
88     $x = $x + 1;
89 }
90
91 // 7. Chamada de função com argumentos inválidos (vírgula sobrando)
92 // Esperava-se uma expressão entre as vírgulas.
93 $y = $soma(2, , 3);
94

```

```

95 // 8. Uso de palavra reservada no lugar errado
96 // 'true' é um valor booleano, não pode ser usado como nome de variável.
97 int true = 5;
98
99 // 9. Esquecer de abrir a chave num bloco de função
100 // O parser espera '{' após a assinatura da função.
101 int $outraFuncao(int $n)
102     return $n;
103 }
104
105 // 10. Esquecer da ',' para declaração de identificadores
106 int $a $b = 5; // Erro (esperado ',' ou ';')
107
108 // 11. Incusão de um '+' após o identificador
109 read($a+); // Erro (esperado ')')
110
111 print("Fim do teste");

```

Listagem 2: Arquivo de teste utilizado

7.2 Saída do Analisador (*stdout*)

1	=====			
2	ANALISE LÉXICA			
3	=====			
4	LIN	COL	LEXEMA	TOKEN
5	-----			
6	3	1	int	TD_INTEGER
7	3	5	\$x	ID_1
8	3	8	=	OP_ATRIBUICAO
9	3	10	-5	NUMERO_INTEIRO
10	3	12	;	DELIMITADOR
11	4	1	float	TD_FLOAT
12	4	7	\$y	ID_2
13	4	10	=	OP_ATRIBUICAO
14	4	12	3.14	NUMERO_FLOAT
15	4	16	;	DELIMITADOR
16	5	1	bool	TD_BOOL
17	5	6	\$ativo	ID_3
18	5	13	=	OP_ATRIBUICAO
19	5	15	true	PR_TRUE
20	5	19	;	DELIMITADOR
21	7	1	int	TD_INTEGER
22	7	5	\$soma	ID_4
23	7	10	(	DELIMITADOR
24	7	11	int	TD_INTEGER
25	7	15	\$a	ID_5
26	7	17	,	DELIMITADOR
27	7	19	int	TD_INTEGER
28	7	23	\$b	ID_6
29	7	25	)	DELIMITADOR
30	7	27	{	DELIMITADOR
31	8	5	int	TD_INTEGER
32	8	9	\$resultado	ID_7
33	8	19	;	DELIMITADOR
34	9	5	\$resultado	ID_7

35	9	16	=	OP_ATRIBUICAO
36	9	18	\$a	ID_5
37	9	21	+	OA_PLUS
38	9	23	\$b	ID_6
39	9	25	;	DELIMITADOR
40	10	5	return	PR_RETURN
41	10	12	\$resultado	ID_7
42	10	22	;	DELIMITADOR
43	11	1	}	DELIMITADOR
44	13	1	bool	TD_BOOL
45	13	6	\$maiorQueZero	ID_8
46	13	19	(	DELIMITADOR
47	13	20	int	TD_INTEGER
48	13	24	\$n	ID_9
49	13	26	)	DELIMITADOR
50	13	28	{	DELIMITADOR
51	14	5	if	PR_IF
52	14	8	(	DELIMITADOR
53	14	9	\$n	ID_9
54	14	12	>	OR_GT
55	14	14	0	NUMERO_INTEIRO
56	14	15	)	DELIMITADOR
57	15	9	return	PR_RETURN
58	15	16	true	PR_TRUE
59	15	20	;	DELIMITADOR
60	16	5	else	PR_ELSE
61	17	9	return	PR_RETURN
62	17	16	false	PR_FALSE
63	17	21	;	DELIMITADOR
64	18	1	}	DELIMITADOR
65	20	1	\$x	ID_1
66	20	4	=	OP_ATRIBUICAO
67	20	6	\$soma	ID_4
68	20	11	(	DELIMITADOR
69	20	12	2	NUMERO_INTEIRO
70	20	13	,	DELIMITADOR
71	20	15	3	NUMERO_INTEIRO
72	20	16	)	DELIMITADOR
73	20	17	;	DELIMITADOR
74	21	1	\$y	ID_2
75	21	4	=	OP_ATRIBUICAO
76	21	6	\$y	ID_2
77	21	9	+	OA_PLUS
78	21	11	1.5	NUMERO_FLOAT
79	21	14	;	DELIMITADOR
80	23	1	if	PR_IF
81	23	4	(	DELIMITADOR
82	23	5	\$x	ID_1
83	23	8	>	OR_GT
84	23	10	3	NUMERO_INTEIRO
85	23	11	)	DELIMITADOR
86	24	5	print	PR_PRINT
87	24	10	(	DELIMITADOR
88	24	11	\$x	ID_1
89	24	13	)	DELIMITADOR
90	24	14	;	DELIMITADOR
91	25	1	else	PR_ELSE
92	26	5	print	PR_PRINT

93	26	10	(	DELIMITADOR
94	26	11	\$y	ID_2
95	26	13	)	DELIMITADOR
96	26	14	;	DELIMITADOR
97	28	1	if	PR_IF
98	28	4	(	DELIMITADOR
99	28	5	\$ativo	ID_3
100	28	12	&&	OL_AND
101	28	15	\$x	ID_1
102	28	18	!=	OR_NE
103	28	21	0	NUMERO_INTEIRO
104	28	22	)	DELIMITADOR
105	28	24	{	DELIMITADOR
106	29	5	print	PR_PRINT
107	29	10	(	DELIMITADOR
108	29	11	"ativo"	LITERAL
109	29	18	)	DELIMITADOR
110	29	19	;	DELIMITADOR
111	30	5	read	PR_READ
112	30	9	(	DELIMITADOR
113	30	10	\$x	ID_1
114	30	12	)	DELIMITADOR
115	30	13	;	DELIMITADOR
116	31	1	}	DELIMITADOR
117	33	1	while	PR_WHILE
118	33	7	(	DELIMITADOR
119	33	8	\$x	ID_1
120	33	11	>	OR_GT
121	33	13	0	NUMERO_INTEIRO
122	33	14	)	DELIMITADOR
123	34	5	\$x	ID_1
124	34	8	=	OP_ATRIBUICAO
125	34	10	\$x	ID_1
126	34	13	-	OA_MINUS
127	34	15	1	NUMERO_INTEIRO
128	34	16	;	DELIMITADOR
129	36	1	while	PR_WHILE
130	36	7	(	DELIMITADOR
131	36	8	\$x	ID_1
132	36	11	<	OR_LT
133	36	13	10	NUMERO_INTEIRO
134	36	15	)	DELIMITADOR
135	36	17	{	DELIMITADOR
136	37	5	\$x	ID_1
137	37	8	=	OP_ATRIBUICAO
138	37	10	\$x	ID_1
139	37	13	+	OA_PLUS
140	37	15	1	NUMERO_INTEIRO
141	37	16	;	DELIMITADOR
142	38	5	print	PR_PRINT
143	38	10	(	DELIMITADOR
144	38	11	\$x	ID_1
145	38	13	)	DELIMITADOR
146	38	14	;	DELIMITADOR
147	39	1	}	DELIMITADOR
148	41	1	if	PR_IF
149	41	4	(	DELIMITADOR
150	41	5	\$x	ID_1

151	41	8	>	OR_GT
152	41	10	0	NUMERO_INTEIRO
153	41	11	)	DELIMITADOR
154	42	5	if	PR_IF
155	42	8	(	DELIMITADOR
156	42	9	\$y	ID_2
157	42	12	>	OR_GT
158	42	14	1.0	NUMERO_FLOAT
159	42	17	)	DELIMITADOR
160	43	9	print	PR_PRINT
161	43	14	(	DELIMITADOR
162	43	15	\$y	ID_2
163	43	17	)	DELIMITADOR
164	43	18	;	DELIMITADOR
165	44	5	else	PR_ELSE
166	45	9	print	PR_PRINT
167	45	14	(	DELIMITADOR
168	45	15	\$x	ID_1
169	45	17	)	DELIMITADOR
170	45	18	;	DELIMITADOR
171	47	1	if	PR_IF
172	47	4	(	DELIMITADOR
173	47	5	\$x	ID_1
174	47	8	==	OR_EQ
175	47	11	0	NUMERO_INTEIRO
176	47	12	)	DELIMITADOR
177	48	5	while	PR_WHILE
178	48	11	(	DELIMITADOR
179	48	12	\$y	ID_2
180	48	15	>	OR_GT
181	48	17	0.0	NUMERO_FLOAT
182	48	20	)	DELIMITADOR
183	49	9	if	PR_IF
184	49	12	(	DELIMITADOR
185	49	13	\$ativo	ID_3
186	49	19	)	DELIMITADOR
187	50	13	print	PR_PRINT
188	50	18	(	DELIMITADOR
189	50	19	"loop"	LITERAL
190	50	25	)	DELIMITADOR
191	50	26	;	DELIMITADOR
192	51	9	else	PR_ELSE
193	52	13	read	PR_READ
194	52	17	(	DELIMITADOR
195	52	18	\$x	ID_1
196	52	20	)	DELIMITADOR
197	52	21	;	DELIMITADOR
198	64	1	\$x	ID_1
199	64	4	=	OP_ATRIBUICAO
200	64	6	\$a	ID_5
201	64	9	+	OA_PLUS
202	64	11	*	OA_MULT
203	64	13	\$b	ID_6
204	64	15	;	DELIMITADOR
205	68	1	\$y	ID_2
206	68	4	=	OP_ATRIBUICAO
207	68	6	(	DELIMITADOR
208	68	7	2	NUMERO_INTEIRO

209	68	9	+	OA_PLUS
210	68	11	3	NUMERO_INTEIRO
211	68	12	;	DELIMITADOR
212	72	1	if	PR_IF
213	72	4	\$x	ID_1
214	72	7	>	OR_GT
215	72	9	0	NUMERO_INTEIRO
216	73	5	print	PR_PRINT
217	73	10	(	DELIMITADOR
218	73	11	\$x	ID_1
219	73	13	)	DELIMITADOR
220	73	14	;	DELIMITADOR
221	77	1	int	TD_INTEGER
222	77	5	\$funcaoErro	ID_10
223	77	16	(	DELIMITADOR
224	77	17	int	TD_INTEGER
225	77	21	\$param1	ID_11
226	77	29	int	TD_INTEGER
227	77	33	\$param2	ID_12
228	77	40	)	DELIMITADOR
229	77	42	{	DELIMITADOR
230	78	5	return	PR_RETURN
231	78	12	0	NUMERO_INTEIRO
232	78	13	;	DELIMITADOR
233	79	1	}	DELIMITADOR
234	83	1	5	NUMERO_INTEIRO
235	83	3	=	OP_ATRIBUICAO
236	83	5	\$x	ID_1
237	83	7	;	DELIMITADOR
238	87	1	while	PR_WHILE
239	87	7	(	DELIMITADOR
240	87	8	)	DELIMITADOR
241	87	10	{	DELIMITADOR
242	88	5	\$x	ID_1
243	88	8	=	OP_ATRIBUICAO
244	88	10	\$x	ID_1
245	88	13	+	OA_PLUS
246	88	15	1	NUMERO_INTEIRO
247	88	16	;	DELIMITADOR
248	89	1	}	DELIMITADOR
249	93	1	\$y	ID_2
250	93	4	=	OP_ATRIBUICAO
251	93	6	\$soma	ID_4
252	93	11	(	DELIMITADOR
253	93	12	2	NUMERO_INTEIRO
254	93	13	,	DELIMITADOR
255	93	15	,	DELIMITADOR
256	93	17	3	NUMERO_INTEIRO
257	93	18	)	DELIMITADOR
258	93	19	;	DELIMITADOR
259	97	1	int	TD_INTEGER
260	97	5	true	PR_TRUE
261	97	10	=	OP_ATRIBUICAO
262	97	12	5	NUMERO_INTEIRO
263	97	13	;	DELIMITADOR
264	101	1	int	TD_INTEGER
265	101	5	\$outraFuncao	ID_13
266	101	17	(	DELIMITADOR

```

267 101 18  int          TD_INTEGER
268 101 22  $n          ID_9
269 101 24  )          DELIMITADOR
270 102 5   return      PR_RETURN
271 102 12  $n          ID_9
272 102 14  ;          DELIMITADOR
273 103 1   }          DELIMITADOR
274 106 1   int          TD_INTEGER
275 106 5   $a          ID_5
276 106 8   $b          ID_6
277 106 11  =          OP_ATRIBUICAO
278 106 13  5          NUMERO_INTEIRO
279 106 14  ;          DELIMITADOR
280 109 1   read       PR_READ
281 109 5   (          DELIMITADOR
282 109 6   $a          ID_5
283 109 8   +          OA_PLUS
284 109 9   )          DELIMITADOR
285 109 10  ;          DELIMITADOR
286 111 1   print      PR_PRINT
287 111 6   (          DELIMITADOR
288 111 7   "Fim do teste" LITERAL
289 111 21  )          DELIMITADOR
290 111 22  ;          DELIMITADOR

```

291 292 TABELA DE SÍMBOLOS

293	=====				
294	ID	HASH	LEXEMA	TOKEN	QUANT. OCORRENCIAS
295	-----				
296	5	1	\$a	ID	5
297	6	2	\$b	ID	4
298	13	6	\$outraFuncao	ID	1
299	9	14	\$n	ID	4
300	1	24	\$x	ID	23
301	2	25	\$y	ID	9
302	7	44	\$resultado	ID	3
303	3	56	\$ativo	ID	3
304	8	71	\$maiorQueZero	ID	1
305	4	82	\$soma	ID	3
306	11	95	\$param1	ID	1
307	12	96	\$param2	ID	1
308	10	99	\$funcaoErro	ID	1

309 310 311 ANALISE SINTATICA

312	=====		
313	[Lin:Col]	ACAO	DETALHE
314	-----		
315	[064:012]	ERRO	syntax error, unexpected *
316	[068:013]	ERRO	syntax error, unexpected ';', expecting ')'
317	[072:006]	ERRO	syntax error, unexpected identificador, expecting '('
318	[077:032]	ERRO	syntax error, unexpected int, expecting ')'
319	[079:002]	ERRO	syntax error, unexpected '}', expecting end of file
320	[087:009]	ERRO	syntax error, unexpected ')'
321	[089:002]	ERRO	syntax error, unexpected '}', expecting end of file
322	[097:009]	ERRO	syntax error, unexpected true, expecting identificador
323	[102:011]	ERRO	syntax error, unexpected return, expecting '{'
324	[103:002]	ERRO	syntax error, unexpected '}', expecting end of file

```

325 [109:009]      ERRO      syntax error, unexpected +, expecting ')'
```

```

326 -----
```

```

327 Analise concluída com 11 erro(s).
```

```

328 =====
```

Listagem 3: Saída gerada pelo analisador léxico

8 Conclusão

A etapa de implementação do analisador sintático utilizando *Bison* permitiu consolidar os conceitos teóricos da análise sintática e aplicá-los à minilinguagem didática definida anteriormente. Durante o desenvolvimento, foi possível observar como decisões de projeto, como a definição de *tokens*, o uso de *yylval* para armazenar valores semânticos relevantes e a separação entre comandos abertos e fechados, impactam diretamente na robustez e clareza do compilador.

O analisador desenvolvido reconhece construções válidas da linguagem, incluindo declarações, expressões, comandos condicionais e de repetição, e permite a recuperação de erros sintáticos por meio do mecanismo de *panic mode*. Conflitos clássicos, como o *dangling else*, foram tratados adequadamente, garantindo consistência estrutural do código e fornecendo mensagens claras ao usuário.

Além disso, esta implementação estabelece a base para as etapas semânticas subsequentes, pois organiza corretamente a árvore de derivação e assegura que os programas válidos estejam estruturados segundo a gramática da linguagem. A experiência adquirida evidencia a importância da integração entre o analisador léxico e sintático, a necessidade de decisões técnicas fundamentadas e a relevância de um projeto modular e extensível.

Em síntese, a etapa alcançou seus objetivos, transformando conceitos teóricos em uma ferramenta funcional, consolidando boas práticas de desenvolvimento de compiladores e oferecendo uma base confiável para análises futuras de tipo, escopo e semântica da minilinguagem.

9 Referências

- [1] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Pearson/Addison-Wesley, 2006.
- [2] TERRA, Ricardo. *Compiladores*. Apostila/slides da disciplina GCC130, Universidade Federal de Lavras (UFLA), 2026.
- [3] OPENAI. *ChatGPT*. Ferramenta de inteligência artificial utilizada como apoio para revisão textual, organização do relatório e esclarecimentos conceituais. Acesso em: 2026.
- [4] ANTHROPIC. *Claude AI*. Ferramenta de inteligência artificial utilizada para organização textual e auxílio na elaboração do relatório em $\text{\LaTeX}$. Acesso em: 2026.